

React Frontend Framework Cheat Sheet

24 tables · 260 concepts · Updated 2026-09-27 · [cheatgrid.com](#)

React is a **declarative, component-based JavaScript library** for building user interfaces, maintained by Meta. It uses a **virtual DOM and Fiber reconciliation engine** to efficiently update only what changes in the actual DOM. React 19, the most significant release since React 18, introduced an action-based async model, Server Components, and new hooks (`useActionState`, `useFormStatus`, `useOptimistic`) that simplify data mutation and form handling. The **React Compiler** (v1.0, stable since October 2025) now automatically memoizes components at build time, making most manual `useMemo` and `useCallback` unnecessary in new code.

CONTENTS

Component Fundamentals	2
JSX and Rendering	3
Core Hooks	3
State Management	5
Rendering Model and Rules of React	6
Event Handling	7
Forms and Validation	8
Effects and Lifecycle	9
Refs and DOM Access	10
Context and Global State	11
Component Patterns	12
Error Handling	13
Performance Optimization	13
React Compiler and Hooks Lints	14
TypeScript Integration	15
Testing	16
DevTools and Debugging	17
Routing (React Router)	17
Styling Approaches	19
Build Tools and Ecosystem	19
State Management Libraries	20
Accessibility Best Practices	21
Advanced React Features	22
Server Components and Server Rendering	22
References	24

TABLE 1

Component Fundamentals

Every React UI starts as function components: small functions that take props and return JSX, composed into a tree and mounted into the DOM with `createRoot` . Props flow down, children nest inside, and the legacy class form still shows up in older code.

Concept	Example	Description
Functional Component	<pre>function Button({ text }) { return <button>{text}</button> }</pre>	• Pure function that accepts props and returns JSX • preferred over class components since hooks in React 16.8.
Props	<pre><UserCard name="Alice" age={30} /></pre>	• Read-only data passed from parent to child • treated as function arguments enabling component reusability.
Children prop	<pre><Card> <h1>Title</h1> </Card></pre>	• Special prop containing nested JSX elements between opening and closing tags • enables composition patterns.
Default props	<pre>function Btn({ type = 'button' }) {}</pre>	• Fallback values for props using ES6 default parameters • replaced older <code>Component.defaultProps</code> static property.
Prop destructuring	<pre>function User({ name, email }) {}</pre>	• Extracts props directly in function parameters • supports nested destructuring and renaming.
Component composition	<pre><Layout> <Sidebar /> <Content>Home</Content> </Layout></pre>	• Building complex UIs from simpler components • React's own recommended pattern to reuse code, in place of class inheritance.
Fragment	<pre><> <td>A</td> <td>B</td> </></pre>	• Returns multiple elements without extra DOM nodes • shorthand <code><></code> or explicit <code><React.Fragment></code> with key support.
Importing and Exporting Components	<pre>export default function Gallery() {} import Gallery from './Gallery.js';</pre>	• Default export gives one component per file, imported under any name • named exports allow several per file.
createRoot	<pre>import { createRoot } from 'react-dom/client'; const root = createRoot(document.getE lementById('root')); root.render(<App />);</pre>	• Creates the root that mounts a React tree into a browser DOM node • an app's entry point, called once, then updated with <code>root.render</code> .
Class component (legacy)	<pre>class Greeting extends Component { render() { return <h1>Hello, {this.props.name}!</h1> } }</pre>	• Base class for defining a component with a <code>render()</code> method • superseded by function components and Hooks in new code
PureComponent (legacy)	<pre>class Greeting extends PureComponent { render() { return <h1>Hello, {this.props.name}!</h1> } }</pre>	• Class component that skips re-rendering when props and state are shallowly equal • the function-component equivalent is <code>React.memo</code> .

TABLE 2

JSX and Rendering

JSX is where JavaScript and markup meet. Curly braces drop expressions into your tree, and plain JavaScript handles the logic that templating languages bolt on with special syntax. These patterns cover the moves you reach for constantly: conditional rendering, mapping lists with stable keys, applying styles and classes, and escape hatches like portals and `dangerouslySetInnerHTML`.

Pattern	Example	Description
JSX rules	<pre><> <h1>Title</h1> </></pre>	- Return a single root element (or a Fragment) - close every tag, including self-closing ones like <code></code> - write attributes in camelCase (<code>className</code> , <code>strokeWidth</code>).
JSX expression	<pre><h1>Hello {user.name}</h1></pre>	- Embeds JavaScript expressions in markup using curly braces - evaluates to React elements at runtime.
Conditional rendering	<pre>{isLoggedIn ? <Dashboard /> : <Login />}</pre>	- Renders elements based on conditions using ternary operator, <code>&&</code> , or if statements - no special template syntax.
List rendering	<pre>{items.map(item => <Item key={item.id} {...item} />)}</pre>	- Transforms arrays into element lists using <code>map()</code> - requires unique key prop for reconciliation optimization.
Key prop	<pre><li key={user.id}>{user.name}</pre>	- Lets React match each array item to its component across renders - must be unique among siblings and never generated during render.
className	<pre><div className="btn btn-primary">Save</div></pre>	Sets CSS classes using <code>className</code> , since <code>class</code> is a reserved word in JavaScript.
Inline styling	<pre><div style={{ color: 'red', fontSize: 16 }}>Alert</div></pre>	- Applies styles as a JavaScript object with camelCased properties - numeric values auto-append <code>px</code> unless the property is unitless.
Document Metadata	<pre><title>My Page</title> <meta name="description" content="..." /></pre>	React 19: <code><title></code> and <code><meta></code> tags rendered anywhere in the component tree are automatically hoisted to <code><head></code> .
Linking external resources	<pre>return (<link rel="stylesheet" href="styles.css" precedence="default" />);</pre>	- React 19: <code><link></code> , <code><script></code> and <code><style></code> rendered anywhere in the tree are hoisted to <code><head></code> and de-duplicated <code>precedence</code> orders stylesheets, <code>async</code> opts a <code><script></code> into hoisting.
dangerouslySetInnerHTML	<pre>const markup = '<p>Hi</p>'; <div dangerouslySetInnerHTML={{ __html: markup }} /></pre>	- Injects raw HTML directly - named to highlight XSS risk, always sanitize untrusted content before use.
Portal	<pre>createPortal(<Modal />, document.body)</pre>	- Renders children into a DOM node outside the parent hierarchy - useful for modals and tooltips, events still bubble through the React tree.

TABLE 3

Core Hooks

Hooks are the functions that let a function component remember state, run side effects, and tap into React's machinery without a class. The ones here range from the everyday workhorses like `useState` , `useEffect` , and `useContext` to React 19's action-era additions like `useActionState` and `useOptimistic` that wire async work directly into your UI. If you only memorize one table on this sheet, make it this one.

Hook	Example	Description
useState	<pre>const [count, setCount] = useState(0)</pre>	- Adds local state to functional components - returns current value and updater function that triggers re-render.

Hook	Example	Description
useEffect	<pre>useEffect(() => { fetchData() }, [id])</pre>	- Runs side effects after render - empty array runs once on mount, deps trigger on changes, cleanup via return function.
useContext	<pre>const theme = useContext(ThemeContext)</pre>	- Consumes Context values without wrapping in Consumer - subscribes to nearest Provider above in tree.
useRef	<pre>const inputRef = useRef(null)</pre>	- Creates mutable reference that persists across renders without triggering updates - commonly used for DOM access.
useReducer	<pre>const [state, dispatch] = useReducer(reducer, init)</pre>	- Alternative to useState for complex state logic - accepts reducer <code>(state, action) => newState</code> following Redux pattern.
useMemo	<pre>const value = useMemo(() => compute(a, b), [a, b])</pre>	Memoizes computed values to avoid expensive recalculations - only recomputes when dependencies change.
useCallback	<pre>const handler = useCallback(() => fn(id), [id])</pre>	Memoizes function identity between renders - prevents child re-renders when passing callbacks as props.
useActionState	<pre>const [state, formAction, isPending] = useActionState(fn, init)</pre>	- React 19 hook that ties an async action to state - returns current state, a dispatchable action ref, and pending flag; eliminates manual loading/error <code>useState</code> boilerplate.
useFormStatus	<pre>const { pending } = useFormStatus()</pre>	- React 19 hook that reads the parent form's submission state - must be called from a component inside a <code><form></code> ; provides <code>pending</code> , <code>data</code> , <code>method</code> , <code>action</code> .
useOptimistic	<pre>const [optimistic, addOptimistic] = useOptimistic(state, fn)</pre>	- Shows optimistic UI update immediately before async action completes - automatically reverts on error.
useTransition	<pre>const [isPending, startTransition] = useTransition()</pre>	- Marks state updates as non-urgent transitions - React 19 supports async functions; keeps UI responsive by deferring or pausing low-priority work.
useDeferredValue	<pre>const deferredValue = useDeferredValue(searchQuery)</pre>	- Returns deferred version of value that lags behind during urgent updates - useful for expensive rendering like filtered lists.
useLayoutEffect	<pre>useLayoutEffect(() => { measure() }, [])</pre>	- Synchronous version of useEffect that fires before browser paint - use for DOM measurements to prevent visual flicker.
useId	<pre>const id = useId()</pre>	- Generates unique stable IDs for accessibility attributes - ensures matching IDs between server and client in SSR.
useEffectEvent	<pre>const onConnected = useEffectEvent(() => { showNotification('Connected!', theme) })</pre>	- Creates a non-reactive callback that always reads the latest props and state - safe to call inside an Effect without adding it to the dependency array.
useSyncExternalStore	<pre>const isOnline = useSyncExtern alStore(subscribe, getSnapshot)</pre>	- Subscribes a component to an external store outside React's own state - used by state-management libraries to read consistent values across concurrent renders.
useInsertionEffect	<pre>useInsertionEffect(() => { // inject <style> tags here })</pre>	- Fires before layout Effects, letting a library inject dynamic CSS before the DOM is read - used almost exclusively by CSS-in-JS library authors.
useDebugValue	<pre>useDebugValue(isOnline ? 'Online' : 'Offline')</pre>	Adds a readable label for a custom Hook, shown in React DevTools.

TABLE 4

State Management

State is what makes a UI interactive, and most state bugs come from putting it in the wrong place or mutating it directly. These techniques are the built-in toolkit before you reach for any library: local `useState`, updating objects and arrays without mutation, controlled vs uncontrolled inputs, reducers, batching, and the judgment calls of lifting, colocating and structuring state.

Technique	Example	Description
Local state	<pre>const [name, setName] = useState('')</pre>	- State scoped to single component using <code>useState</code> - re-renders component when updated.
State updater function	<pre>setCount(prev => prev + 1)</pre>	- Passes function to setter instead of value - ensures update uses most recent state, critical in async contexts.
Updating objects in state	<pre>setPerson({ ...person, artwork: { ...person.artwork, city: 'New Delhi' } });</pre>	- Treat state objects as read-only, copy with <code>...</code> spread instead of mutating a field - a nested object needs one spread per level.
Updating arrays in state	<pre>setArtists(artists.filter(a => a.id !== id))</pre>	- Use non-mutating methods (<code>map</code>, <code>filter</code>, <code>[...arr]</code>) instead of <code>push</code>, <code>splice</code> or <code>pop</code> - produces a new array so React can detect the change.
Controlled component	<pre><input value={text} onChange={e => setText(e.target.value)} /></pre>	- Form element whose value is controlled by React state - value prop + onChange handler create single source of truth.
Uncontrolled component	<pre><input defaultValue="hi" ref={inputRef} /></pre>	- Form element that manages own state via DOM - access value through refs instead of state.
Reducer pattern	<pre>const reducer = (state, action) => { switch(action.type) {...} }</pre>	- Centralizes complex state logic in pure reducer function - action objects describe state changes declaratively.
State batching	<pre>setCount(c => c + 1) setFlag(f => !f)</pre>	React groups multiple state updates into a single re-render automatically, even inside promises, timeouts and native event handlers.
Derived state	<pre>const fullName = firstName + ' ' + lastName</pre>	- Values computed from props/state during render - avoid storing in separate state to prevent synchronization bugs.
Lifting state up	<pre>const [val, setVal] = useState() <B onChange={setVal} /></pre>	- Move state to nearest common ancestor when multiple siblings need the same data - passed down as props.
Resetting state with a key	<pre>{isPlayerA ? <Counter key="Taylor" /> : <Counter key="Sarah" />}</pre>	- React ties state to a component's position in the render tree, not its identity - changing the <code>key</code> makes React treat it as a new instance and reset its state.
State colocation	<pre>function Panel() { const [open, setOpen] = useState(false) }</pre>	- Place state closest to where it's used - limits re-render scope and opposes premature lifting.
Avoid duplicated state	<pre>const [selectedId, setSelectedId] = useState(0) const selectedItem = items.find(i => i.id === selectedId)</pre>	- Store the minimal essential data (an id) instead of duplicating an object already held elsewhere - derive the rest so nothing can fall out of sync.

TABLE 5

Rendering Model and Rules of React

How React actually renders your UI and the rules code must follow for that mental model to hold: where Hooks may be called, what "pure" means during render, and why a state variable never changes mid-render. Breaking one of these is the source of most beginner React bugs.

Concept	Example	Description
Rules of Hooks	<pre>// • inside a condition if (cond) { useState(0); } // ✓ top level, before any return function Comp() { const [x] = useState(0); }</pre>	• Call Hooks only at the top level of a component or custom Hook, never in a loop, condition or after an early return • only from a component or custom Hook, never a plain function
State as a Snapshot	<pre>const [n, setN] = useState(0); function onClick() { setN(n + 1); console.log(n); // still the old value }</pre>	• Setting state does not change the variable in the current render, it schedules a re-render • every event handler keeps the state values from the render it was created in
Render and Commit	<pre>const root = createRoot(el); root.render(<App />); // 1. trigger // 2. render: React calls App() // 3. commit: React updates the DOM</pre>	• Trigger: initial mount or a state update queues a render • Render: React calls the component functions • Commit: React applies only the DOM changes that differ from last time
Components and Hooks Must Be Pure	<pre>// • impure: differs on every call function Clock() { return new Date().toString(); } // ✓ pure: same inputs, same output function Sum({ a, b }) { return a + b; }</pre>	• Given the same props, state and context a component must always return the same JSX • no side effects in render, and props and state are never mutated
Local Mutation	<pre>function FriendList({ friends }) { const items = []; // created in this render friends.forEach(f => items.push(<Friend key={f.id} friend={f} />)); return <section>{items}</section>; }</pre>	• Mutating a value created during the same render is fine • mutating a value that existed before it breaks purity
React Calls Components and Hooks	<pre>// ✓ let React call it, via JSX <BlogPost /> // • called directly, skips React {BlogPost()}</pre>	• Never call a component function directly or pass a Hook around as a value • only React invokes them, which is how it enforces the Rules of Hooks
Your UI as a Tree	<pre>// render tree <App> <FancyText /> <InspirationGenerator> <Copyright /> </InspirationGenerator> </App></pre>	• The render tree models parent and child components for one render pass • the module dependency tree models import statements, used by bundlers
Strict Mode	<pre>// wrap the app root <StrictMode> <App /> </StrictMode></pre>	• Development-only tool that highlights potential problems • intentionally double-invokes functions to surface side effects.

TABLE 6

Event Handling

React wraps native browser events in a cross-browser event object and attaches a single listener at the root rather than wiring up handlers on every node. The entries here cover the everyday mechanics: passing handlers, reading the event object, controlling propagation, and the inline-versus-named-handler tradeoff.

Concept	Example	Description
Event handler	<code><button onClick={handleClick}>Click</button></code>	- Function passed to event prop - convention names handlers <code>handleEvent</code> or <code>onEvent</code> .
Event object	<pre>function handle(e) { e.preventDefault() console.log(e.target.value) }</pre>	SyntheticEvent instance with <code>target</code> , <code>currentTarget</code> - methods <code>preventDefault()</code> , <code>stopPropagation()</code> .
Synthetic events	<code>onClick={e => console.log(e.type)}</code>	- React's cross-browser wrapper, also called a synthetic event - conforms to the DOM standard but fixes browser inconsistencies.
Preventing default behavior	<code><form onSubmit={e => { e.preventDefault() submit() }}></code>	- Stops the browser's built-in behavior for the event, such as a form submit reloading the page - unrelated to <code>stopPropagation()</code> , which stops handlers instead of the default action.
Event propagation	<pre><div onClick={() => alert('parent')}> <button onClick={() => alert('child')}>Click</button> </div></pre>	- Events bubble up the tree from the target to its ancestors - call <code>e.stopPropagation()</code> in any handler to stop it climbing further.
Capture phase (onClickCapture)	<pre><div onClickCapture={() => log('first')}> <button onClick={handleClick} /> </div></pre>	- Add <code>Capture</code> to an event name to run the handler on the way down, before the target's own handler - useful for routers or analytics that must run regardless of propagation.
Passing an argument	<code><button onClick={() => deleteRow(id)}>Delete</button></code>	- Wrap the call in an arrow function to bind extra parameters to a handler - calling the function directly in JSX, without the arrow wrapper, runs it immediately during render.
Named handler	<pre>const handleClick = () => {...} <button onClick={handleClick}></pre>	Declaring function separately then passing reference - better for reusability and testing than inline.
Inline handler	<code><button onClick={() => alert('clicked')}></code>	- Defining function directly in JSX - convenient for simple logic but creates new function each render.
Event delegation	React attaches all handlers to the root container ; events bubble up to a single listener for performance.	- React 17+ attaches to root instead of document - improves micro-frontend isolation and compatibility.
Custom events	<pre>const event = new CustomEvent('build', { detail: elem.dataset.time }) elem.dispatchEvent(event)</pre>	Dispatch native custom events with the <code>CustomEvent</code> constructor, then read them via <code>addEventListener</code> on a ref's DOM node.

TABLE 7

Forms and Validation

Forms are where state, events, and validation all collide. The classic React approach is the controlled input, its value driven by state and updated on every change, but React 19's form actions now handle pending state, errors, and resets back to the framework. This table walks each control type plus when a library like React Hook Form is worth the dependency.

Technique	Example	Description
Controlled input	<pre><input value={name} onChange={e => setName(e.target.value)} /></pre>	- Input value controlled by state - onChange handler updates state creating bidirectional binding.
Form submission	<pre><form onSubmit={e => { e.preventDefault() submit(data) }}></pre>	Intercepting submit event to prevent default page reload and handle data programmatically.
Form Actions	<pre>async function fn(prev, data) { const email = data.get('email') } const [state, formAction] = useActionState(fn, null) <form action={formAction}></pre>	- React 19: native action prop accepts an async function - React manages pending state, errors, and resets automatically.
Reading form values with FormData	<pre>function handleSubmit(e) { e.preventDefault() const data = new FormData(e.target) const email = data.get('email') }</pre>	- Collects every named field in one call, keeping inputs uncontrolled - Read a field with .get(name) instead of tracking it in state.
Multiple submit buttons (formAction)	<pre><form action={publish}> <button type="submit">Publish</button> <button formAction={save}>Save draft</button> </form></pre>	Each button can override the form's action with its own formAction , so one form supports several submission behaviors .
Checkbox	<pre><input type="checkbox" checked={agreed} onChange={e => setAgreed(e.target.checked)} /></pre>	- Uses checked prop and e.target.checked for value - controlled with boolean state.
Select box	<pre><select value={selected} onChange={e => setSelected(e.target.value)}> <option value="a">A</option> </select></pre>	- Uses value on <select> not selected on <option> single source of truth pattern.
Textarea	<pre><textarea value={text} onChange={e => setText(e.target.value)} /></pre>	- Uses value prop like inputs (not children) - controlled the same way for consistency.
File input	<pre>const fileRef = useRef(null) <input type="file" ref={fileRef} /></pre>	- Always uncontrolled because value is read-only - access files via fileRef.current.files.
Form libraries	<pre>const { register, handleSubmit } = useForm()</pre>	Libraries like React Hook Form or Formik reduce boilerplate for validation, submission, and field management.
Validation	<pre>const error = email && !email.includes('@') ? 'Invalid email' : null</pre>	- Implementing inline validation by checking state values - display errors conditionally based on validation rules.

TABLE 8

Effects and Lifecycle

Effects are how a component stays in sync with the outside world: timers, subscriptions, event listeners, and data fetches that live beyond React's render. The recurring theme is the cleanup function, tearing down before the next run or on unmount, and the discipline to reach for an Effect only when there really is an external system to synchronize with.

Pattern	Example	Description
Effect with cleanup	<pre>useEffect(() => { const id = setInterval(tick, 1000) return () => clearInterval(id) }, [])</pre>	- Effect returning cleanup function runs before unmount or before next effect - prevents memory leaks.
Dependency array	<pre>useEffect(() => fetch(url), [url])</pre>	Controls when the effect runs: empty array runs once on mount, omitted runs every render, with dependencies runs only when they change.
You Might Not Need an Effect	<pre>// during render, no Effect needed const fullName = firstName + ' ' + lastName</pre>	- Effects are for syncing with an external system, not for deriving state from props or other state - a value you can calculate during render should be calculated there.
Custom Hook	<pre>function useOnlineStatus() { const [isOnline, setIsOnline] = useState(true) useEffect(() => { window.addEventListener('online', () => setIsOnline(true)) window.addEventListener('offline', () => setIsOnline(false)) }, []) return isOnline }</pre>	- Extracts reusable stateful logic into a function prefixed with "use" - can call other Hooks internally.
Data fetching	<pre>useEffect(() => { fetchBio(person).then(result => setBio(result)) }, [person])</pre>	- Loading external data in an effect - consider React Query or SWR in production to handle caching and race conditions.
Fetching race condition	<pre>useEffect(() => { let ignore = false fetchResults(query, page).then(json => { if (!ignore) setResults(json) }) return () => { ignore = true } }, [query, page])</pre>	- Two overlapping requests can resolve out of order - an <code>ignore</code> flag (or <code>AbortController</code>) in the cleanup discards the stale response.
Event listeners	<pre>useEffect(() => { window.addEventListener('resize', handle) return () => window.removeEventListener('resize', handle) }, [])</pre>	- Attaching global event handlers outside React's event system - cleanup prevents duplicate listeners.
Interval/Timeout	<pre>useEffect(() => { const id = setInterval(() => { onTick() }, delay) return () => clearInterval(id) }, [delay])</pre>	- Running timed operations inside an effect - cleanup cancels the pending timer so it never fires after unmount or a dependency change.
Reconnecting on dependency change	<pre>useEffect(() => { const connection = createConnection(serverUrl, roomId) connection.connect() return () => connection.disconnect() }, [roomId])</pre>	- When a dependency changes, React cleans up the old effect before running the new one - the effect reconnects to the external system with the fresh value.

Pattern	Example	Description
Removing Effect dependencies	<pre>useEffect(() => { const options = { serverUrl, roomId } const connection = createConnection(options) connection.connect() return () => connection.disconnect() }, [roomId])</pre>	- An object or function recreated every render looks like a changed dependency even when its values did not change - move its creation inside the effect instead of adding it to the array.
Separating events from Effects	<pre>const onConnected = useEffectEvent(() => { if (!muted) showNotification('Connected!') }) useEffect(() => { const connection = createConnection(roomId) connection.on('connected', onConnected) connection.connect() return () => connection.disconnect() }, [roomId])</pre>	<code>useEffectEvent</code> wraps non-reactive logic that reads the latest props and state without becoming a dependency - a stable Hook imported from <code>react</code>, not <code>experimental_useEffectEvent</code>.
Class lifecycle to Hooks	<pre>// class: componentDidMount + componentDidUpdate + componentWillUnmount useEffect(() => { const connection = createConnection(serverUrl, roomId) connection.connect() return () => connection.disconnect() }, [roomId, serverUrl])</pre>	- A single <code>useEffect</code> call replaces <code>componentDidMount</code>, <code>componentDidUpdate</code> and <code>componentWillUnmount</code> together - its cleanup function plays the role <code>componentWillUnmount</code> used to play.

TABLE 9

Refs and DOM Access

Refs are React's escape hatch to the actual DOM and to mutable values that shouldn't trigger re-renders. Reach for them when you need to focus an input, measure a node, or hold a value like a timer ID across renders. React 19 simplified the area: ref is now a regular prop and ref callbacks can return cleanup functions, retiring the old `forwardRef` ceremony.

Concept	Example	Description
useRef	<pre>const inputRef = useRef(null) <input ref={inputRef} /></pre>	- Creates mutable reference to DOM element or value <code>current</code> persists without causing re-renders.
DOM manipulation	<pre>inputRef.current.focus()</pre>	- Directly calling DOM methods via ref - escape hatch for imperative operations React doesn't provide.
Storing values	<pre>const countRef = useRef(0) countRef.current++</pre>	- Using ref to hold mutable value that doesn't trigger re-renders - useful for intervals, previous values, instance variables.
Callback ref	<pre><div ref={node => setNode(node)} /></pre>	- Ref as function called with DOM node on mount and <code>null</code> on unmount - useful for measuring or observing nodes.
Ref cleanup	<pre><div ref={node => { setup(node) return () => cleanup(node) }} /></pre>	- React 19: ref callbacks can return a cleanup function called when element unmounts - replaces manual null check pattern.
Refs in loops	<pre>const itemsRef = useRef(new Map()) <li ref={node => itemsRef.current.set(id, node)} /></pre>	- Storing multiple refs in a Map or array - ref callback pattern manages dynamic lists of refs.
Ref as prop	<pre>function Input({ ref }) { return <input ref={ref} /> }</pre>	- React 19: ref is a regular prop in function components - no <code>forwardRef</code> wrapper needed, simpler and cleaner.

Concept	Example	Description
useImperativeHandle	<pre>useImperativeHandle(ref, () => ({ focus: () => inputRef.current.focus() }))</pre>	- Customizes instance value exposed to parent via ref - controls what methods/properties are public.
flushSync	<pre>flushSync(() => { setTodos([...todos, newTodo]) }) listRef.current.lastChild.scrollIntoView()</pre>	- Forces React to flush a state update synchronously before the next line runs - escape hatch to read or scroll to fresh DOM right after updating, use sparingly.
Fragment refs	<pre><Fragment ref={fragmentRef}> <Child /> </Fragment> fragmentRef.current.focus()</pre>	- React 19.3: pass ref to <code><Fragment></code> (only the explicit tag, not <code><></code>) - returns a FragmentInstance with <code>focus</code> , <code>scrollIntoView</code> , <code>addEventListener</code> for its first-level DOM children.
forwardRef (legacy)	<pre>const Comp = forwardRef((props, ref) => <input ref={ref} />)</pre>	- Legacy pattern passing ref from parent through component to child - superseded by ref as a prop, React plans to deprecate and remove it.
createRef (legacy)	<pre>class Form extends Component { inputRef = useRef() handleClick = () => this.inputRef.current.focus() }</pre>	Class-component way to create a ref object - unlike <code>useRef</code> , returns a new object on every call; use <code>useRef</code> in function components.

TABLE 10

Context and Global State

Context solves prop drilling: passing data straight to deep consumers without threading it through every layer in between. These entries go from `createContext` to the Context-plus- `useReducer` combo that acts as a lightweight Redux for simpler apps, plus the memoization tricks that stop a Provider from re-rendering every consumer on each change.

Pattern	Example	Description
createContext	<pre>const ThemeContext = createContext('light')</pre>	- Creates Context object with default value - default used when no Provider exists above in tree.
Context Provider	<pre><ThemeContext value="dark"> <App /> </ThemeContext></pre>	- In React 19, the context object itself is rendered as a Provider <code><ThemeContext.Provider></code> is the legacy pre-19 form
useContext	<pre>const theme = useContext(ThemeContext)</pre>	- Reads context value from nearest Provider above - replaces older <code>Context.Consumer</code> pattern.
use(Context)	<pre>function Button({ show }) { if (show) { const theme = use(ThemeContext) return <hr className={theme} /> } }</pre>	- Reads context like <code>useContext</code> , but can be called conditionally or inside loops <code>useContext</code> must run at the top level, <code>use</code> does not.
Context with reducer	<pre>const [state, dispatch] = useReducer(reducer, init) <Ctx value={{state, dispatch}}></pre>	- Combining Context + useReducer for global state management - lightweight alternative to Redux for simpler apps.
Custom provider	<pre>function TasksProvider({ children }) { const [tasks, dispatch] = useReducer(tasksReducer, initialTasks) return <TasksContext value={tasks}>{children}</TasksContext> }</pre>	- Wraps a Provider in a component with its own logic - encapsulates state management behind a clean import.

Pattern	Example	Description
Overriding context	<pre><ThemeContext value="dark"> <ThemeContext value="light"> <Footer /> </ThemeContext> </ThemeContext></pre>	- Wrapping part of the tree in another Provider overrides the value for components inside it - providers can be nested and overridden as many times as needed.
Context performance	<pre>const contextValue = useMemo(() => ({ currentUser, login }), [currentUser, login])</pre>	Memoizing the context value stops it being a new object on every render - prevents consumers re-rendering when the underlying data has not changed.

TABLE 11

Component Patterns

These are the reusable shapes experienced React developers reach for to keep components flexible and DRY. Some of them, like the provider pattern, compound components, and composition through children, are alive and well. Others like render props and higher-order components have largely been replaced by custom hooks, so this table doubles as a map of which patterns still earn their keep.

Pattern	Example	Description
Composition	<pre><Card> {children} </Card></pre>	- Building components by combining smaller ones - uses the <code>children</code> prop to accept nested content flexibly.
Provider pattern	<pre><ThemeProvider> <App /> </ThemeProvider></pre>	- Wraps tree with Context Provider to share values - avoids prop drilling for global concerns like theme or auth.
Compound components	<pre><Tabs> <Tabs.List> <Tabs.Tab /> </Tabs.List> </Tabs></pre>	- Components that work together sharing implicit state - creates a cohesive API like native HTML <code><select></code> and <code><option></code> .
Container/Presentational	<pre>// Container: fetches data // Presentational: renders UI</pre>	- Separates logic from presentation - containers handle state and effects, presentational components are pure render functions.
Slots pattern	<pre><Layout header={<Header />} sidebar={<Sidebar />} /></pre>	- Passes multiple JSX elements as named props - more explicit than <code>children</code> for complex layouts.
Render props	<pre><DataProvider render={data => <Display data={data} />} /></pre>	- Passes a function as a prop that the component calls to determine render output - largely superseded by custom hooks.
Higher-Order Component	<pre>const EnhancedComponent = withSubscription(WrappedComponent)</pre>	- Function that takes a component and returns an enhanced version - adds props or wraps with logic. Less common since hooks and the React Compiler.
Children (legacy)	<pre>Children.map(children, child => <div className="Row">{child}</div>)</pre>	- Utility methods (<code>map</code> , <code>forEach</code> , <code>count</code> , <code>toArray</code>) for the opaque <code>children</code> prop - a legacy API; exposing separate components is usually clearer.
cloneElement (legacy)	<pre>Children.map(children, child => cloneElement(child, { isHighlighted: true }))</pre>	- Creates a copy of an element with new props or children - a legacy API; prefer passing data through props or a render prop.

TABLE 12

Error Handling

A thrown error during render shouldn't take down the whole app, error boundaries catch it and show a fallback instead. The key gotcha worth internalizing: boundaries only catch errors in rendering and lifecycle, not in event handlers or async code, so those still need plain try-catch.

Technique	Example	Description
Error Boundary	<pre>class ErrorBoundary extends React.Component { componentDidCatch(error, info) {...} render() { return this.state.hasError ? <Fallback /> : this.props.children } }</pre>	• Class component that catches JavaScript errors in child tree • no Hook equivalent exists, so it must stay a class.
componentDidCatch	<pre>componentDidCatch(error, errorInfo) { logError(error, errorInfo.componentStack) }</pre>	• Lifecycle for side effects after error is caught • log errors to reporting service while displaying fallback.
getDerivedStateFromError	<pre>static getDerivedStateFromError(error) { return { hasError: true } }</pre>	• Lifecycle method for updating state when error is thrown • used to render fallback in the next render pass.
Error boundary library	<pre><ErrorBoundary FallbackComponent={ErrorFallback}> <App /> </ErrorBoundary></pre>	react-error-boundary provides hooks-based error handling (<code>useErrorBoundary</code>), reset, and retry capabilities.
resetKeys	<pre><ErrorBoundary resetKeys={[userId]} onReset={() => setUserId(null)}> <Profile userId={userId} /> </ErrorBoundary></pre>	• Clears a triggered boundary and retries rendering when the listed values change • safer than remounting on stale data.
Try-catch in handlers	<pre>try { riskyOperation() } catch (error) { setError(error.message) }</pre>	• Manual error handling in event handlers or effects • Error Boundaries do not catch errors in these contexts.
onCaughtError	<pre>createRoot(container, { onCaughtError: (error, info) => report(error), onUncaughtError: (error, info) => report(error), onRecoverableError: (error, info) => report(error), })</pre>	• Root option that replaces React's default console logging when an Error Boundary catches an error • pair with <code>onUncaughtError</code> and <code>onRecoverableError</code> for full coverage.

TABLE 13

Performance Optimization

Most React performance work comes down to rendering less and rendering smaller. With the React Compiler now stable, much of the old manual memoization is handled for you, but knowing what `React.memo`, code splitting, list virtualization, and the concurrent hooks actually do still tells you where to look when a UI feels sluggish.

Technique	Example	Description
React.memo	<pre>const List = React.memo(function List({ items }) {...})</pre>	• Prevents re-renders if props haven't changed using shallow comparison • second argument (<code>arePropsEqual</code>) lets you customize the comparison.
useCallback	<pre>const onClick = useCallback(() => fn(id), [id])</pre>	• Only worth it when the function is passed to a memoized child or used as a Hook dependency • React Compiler handles this automatically when enabled.

Technique	Example	Description
Code splitting	<pre>import { lazy } from 'react'; const Comp = lazy(() => import('./Comp'))</pre>	- Splits bundle into smaller chunks loaded on demand <code>lazy</code> with dynamic <code>import()</code> creates split points.
Suspense	<pre><Suspense fallback=<Spinner />> <LazyComp /> </Suspense></pre>	- Shows fallback UI while lazy component or async data loads - works with <code>React.lazy</code> and data fetching libraries.
List virtualization	<pre><List rowComponent={Row} rowCount={1000} rowHeight={35} /></pre>	- Renders only visible list items using libraries like <code>react-window</code> - dramatically improves performance for long lists.
Production build	<pre>npm run build</pre>	- Creates optimized bundle with minification and dead code elimination - significantly faster than development build.

TABLE 14

React Compiler and Hooks Lints

React Compiler automatically memoizes components and hooks at build time, and `eslint-plugin-react-hooks` is the linter React itself ships to enforce the Rules of React, including the compiler's own diagnostics.

Concept	Example	Description
React Compiler	<pre>// babel-plugin-react-compiler // no code changes required</pre>	- Build-time tool (v1.0, stable Oct 2025) that automatically memoizes components and hooks - makes most manual <code>useMemo</code> / <code>useCallback</code> unnecessary; opt-in via Babel/Vite plugin.
eslint-plugin-react-hooks	<pre>import reactHooks from 'eslint-plugin-react-hooks'; export default [reactHooks.configs .flat.recommended];</pre>	- Official ESLint plugin enforcing the Rules of React - the recommended preset bundles <code>exhaustive-deps</code> , <code>rules-of-hooks</code> and the compiler's own diagnostics, usable without adopting the compiler.
exhaustive-deps	<pre>useEffect(() => { console.log(count); }, []); // ⚠ missing 'count'</pre>	Warns when a value used inside <code>useEffect</code> , <code>useMemo</code> or <code>useCallback</code> is missing from its dependency array, causing stale closures.
compilationMode	<pre>{ compilationMode: 'annotation' // only "use memo" functions }</pre>	Controls which functions the compiler optimizes: <code>infer</code> (default heuristic), <code>annotation</code> , <code>syntax</code> or <code>all</code> .
"use memo" / "use no memo" directives	<pre>function Optimized() { "use memo"; } function Skipped() { "use no memo"; }</pre>	- String directives that force a function into or out of compilation - escape hatches for <code>annotation</code> mode or for excluding incompatible code.
Incremental adoption	<pre>overrides: [{ test: './src/modern/**/*.{js,jsx,ts,tsx}', plugins: ['babel-plugin-react-compiler'] }]</pre>	Rolls the compiler out gradually with Babel directory overrides, the <code>"use memo"</code> opt-in directive, or gating, rather than enabling it everywhere at once.
Gating	<pre>{ gating: { source: 'my-feature-flags', importSpecifierName: 'isCompilerEnabled' } }</pre>	Runtime feature flag that switches between compiled and original code, for A/B testing or a gradual rollout.

Concept	Example	Description
panicThreshold	<pre>{ panicThreshold: 'none' // skip broken components instead of failing the build }</pre>	Decides whether a compilation error fails the whole build or the compiler just skips optimizing that component.
target	<pre>{ target: '18' // requires the react-compiler-runtime package }</pre>	Sets the minimum React version (17, 18 or 19) the compiled output must stay compatible with.
React Compiler diagnostics	<pre>function Component({ value }) { const [count, setCount] = useState(0); setCount(value); // ✖ set-state-in-render }</pre>	• Compiler-era lints on top of the classics: <code>purity</code> , <code>immutability</code> , <code>refs</code> , <code>set-state-in-render</code> , <code>set-state-in-effect</code> , <code>preserve-manual-memoization</code> , <code>incompatible-library</code> • each catches a Rules-of-React violation that blocks safe memoization.
Compiling libraries	<pre>npm install -D babel-plugin-react-compiler@latest // babel.config.js: plugins: ['babel-plugin-react-compiler']</pre>	Lets library authors ship pre-compiled, already-memoized code so consumers benefit without configuring the compiler themselves.

TABLE 15

TypeScript Integration

TypeScript and React are a near-default pairing now, and most of the friction is in typing the few things TypeScript can't infer on its own. These patterns cover the spots that trip people up: typing props, state generics, event handlers, refs, and children, small habits that catch a surprising share of bugs before the code ever runs.

Pattern	Example	Description
Props interface	<pre>interface Props { name: string; age: number } function User(props: Props) {}</pre>	• Separate type definition for props • enables reuse and clearer documentation.
Component typing	<pre>function Button({ text }: { text: string }) {}</pre>	• Inline props type for simple components • clear and concise for few props.
useState typing	<pre>const [user, setUser] = useState<User null>(null)</pre>	• Explicitly typing state generic when TypeScript can't infer • necessary for complex types or null initialization.
Reducer typing	<pre>interface State { count: number } type Action = { type: "reset" } { type: "add"; value: number } useReducer(reducer, initialState)</pre>	• State type is usually set on <code>initialState</code> and inferred from there • a union type describes the possible dispatched actions.
Event typing	<pre>const handle = (e: React.ChangeEvent<HTMLInputElement>) => {}</pre>	• Typing event handlers with specific event types • ensures type safety for <code>e.target</code> properties.
Ref typing	<pre>const ref = useRef<HTMLDivElement>(null)</pre>	• Providing element type for ref • <code>null</code> initial value requires union type for <code>.current</code> .
Children typing	<pre>interface Props { children: React.ReactNode }</pre>	• ReactNode accepts any child: elements, strings, numbers, fragments • ReactElement (close to <code>JSX.Element</code>) is narrower: JSX only, no primitives.
Context typing	<pre>const Ctx = createContext<ContextType undefined>(undefined)</pre>	• Typing Context value and initial value • <code>undefined</code> forces Provider usage with runtime check.
ComponentProps	<pre>type ButtonProps = React.ComponentPr opsWithoutRef<'button'></pre>	• Extracts the prop types of a DOM tag or another component • <code>ComponentPropsWithoutRef</code> drops <code>ref</code> , handy when wrapping a native element.

Pattern	Example	Description
Generic components	<pre>function List<T>(props: { items: T[]; render: (item: T) => ReactNode }) {}</pre>	- Components accepting generic type parameters - enables type-safe reusable components.
FC type	<pre>const Button: React.FC<Props> = ({ text }) => {}</pre>	- Typing component as FunctionComponent - modern React types dropped its implicit <code>children</code> , so it is now mostly a stylistic choice.

TABLE 16

Testing

The modern React testing philosophy is to test what the user does, not how the component is wired internally. React Testing Library leans hard into that by querying through accessible roles and text. Pair it with a runner like Jest or Vitest and the helpers here, and your tests survive refactors instead of breaking on them.

Tool	Example	Description
React Testing Library	<pre>render(<Button />) screen.getByRole('button')</pre>	- Encourages testing user behavior not implementation - queries based on accessibility attributes.
render	<pre>const { container } = render(<App />)</pre>	- Renders a component into a test DOM - returns queries and the container element.
screen queries	<pre>screen.getByText('Submit')</pre>	Built-in query methods like <code>getBy</code> , <code>queryBy</code> , <code>findBy</code> with variants for text, role, label, test ID, etc.
Query Priority	<pre>screen.getByRole('button', { name: /submit/i })</pre>	- Recommended order: <code>getByRole</code> first, then label, placeholder or text - test ID (<code>getByTestId</code>) is the last resort.
userEvent	<pre>await user.click(button)</pre>	- More realistic user interaction simulation than <code>fireEvent</code> - handles focus, timing, and keyboard events properly.
fireEvent	<pre>fireEvent.click(button)</pre>	- Triggers DOM events on elements directly - synchronous, prefer <code>userEvent</code> for realistic interaction.
waitFor	<pre>await waitFor(() => expect(screen.getByText('Loaded')).toBeInTheDocument())</pre>	- Waits for async changes like state updates or API responses - retries until callback stops throwing.
act()	<pre>await act(async () => { root.render(<Counter />) })</pre>	- Flushes pending React updates before assertions <code>render</code> and <code>userEvent</code> already wrap it; use it directly for lower-level test setup.
renderHook	<pre>const { result } = renderHook(() => useLoggedInUser())</pre>	- Renders a custom Hook in a throwaway test component - exposes the latest return value as <code>result.current</code> .
Vitest	<pre>import { test, expect } from 'vitest' test('adds', () => expect(1+1).toBe(2))</pre>	Jest-compatible test runner built on Vite - significantly faster cold starts and watch mode; ideal for Vite-based React projects; easy migration path from Jest (swap <code>jest.fn</code> for <code>vi.fn</code> ; globals are off by default).
Jest	<pre>test('renders title', () => {...})</pre>	Test runner and assertion library - provides <code>describe</code> , <code>test</code> , <code>expect</code> , and mocking capabilities.
Mock functions	<pre>const onClick = jest.fn() expect(onClick).toHaveBeenCalledTimes(1)</pre>	- Creates spy functions to verify calls <code>jest.fn()</code> (or <code>vi.fn()</code> in Vitest) tracks arguments and call count.
Snapshot testing	<pre>expect(tree).toMatchSnapshot()</pre>	- Captures component output and detects unexpected changes - useful for catching regressions in UI structure.

TABLE 17

DevTools and Debugging

When something renders wrong, too often, or not at all, these are the tools that show you why. React DevTools walks the live component tree and profiles renders; the browser's Performance panel adds React-specific tracks alongside the rest of the page's activity.

Tool	Example	Description
React DevTools	Browser extension showing component tree with props, state, hooks; inspect and edit values live.	- Official debugging browser extension for Chrome and Firefox - essential for development.
Components tab	Type a name in the search box to filter the tree, then select a node to inspect its props, state, and hooks.	- Browses the live component hierarchy in React DevTools - layout resizes to the browser window; search narrows results by name.
Profiler tab	Record a session in React DevTools, then step through each commit to see which components rendered and which hooks changed.	- Interactive profiler tab in React DevTools; shows changed props and hooks per commit - the older Timeline profiler tab was removed in DevTools 8.0.0, replaced by the browser's Performance panel.
Profiler	<pre><Profiler id="Nav" onRender={callback}> <Nav /> </Profiler></pre>	- Measures rendering performance programmatically via an <code>onRender</code> callback - profiling is disabled in production builds by default.
React Performance Tracks	Open the browser's Performance panel and record a trace; Scheduler, Render, and Components tracks appear alongside network and JS activity.	- React-specific entries on the browser's Performance panel timeline: Scheduler, Render, and Components tracks - enabled by default in development; profiling builds need the <code>react-dom/profiling</code> alias.
Suspense tab	Open the Suspense tab to see which boundary is showing a fallback and what caused a component to suspend.	- On by default since DevTools 8.0.0 - shows Suspense boundaries and loading state without extra setup
Source maps	<code>devtool: 'source-map'</code> in config	- Maps compiled code to source - enables debugging original TypeScript/JSX in browser despite minification.
captureOwnerStack	<pre>const stack = captureOwnerStack(); console.log(stack);</pre>	- Reads the current Owner Stack in development, as a string or <code>null</code> - used inside error handlers and custom error overlays to log which component created the failing node.
\$r (console)	<pre>\$r.props \$r.state</pre>	Selecting a component in React DevTools exposes it as <code>\$r</code> in the browser console, for inspecting its props, state, and instance properties.

TABLE 18

Routing (React Router)

React ships no router of its own, so React Router is the de facto standard for mapping URLs to components in single-page apps. This covers the core pieces, routers, route definitions, navigation links, and the hooks that read params and location, the data APIs (loaders, actions, fetchers) that power its Data and Framework modes, plus a nod to TanStack Router for teams that want fully type-safe routing.

Component	Example	Description
BrowserRouter	<pre><BrowserRouter> <App /> </BrowserRouter></pre>	- Uses the HTML5 History API for clean URLs - wraps app to enable client-side routing without a hash.
Routes	<pre><Routes> <Route path="/" element={<Home />} /> </Routes></pre>	- Container for route definitions replacing older <code><Switch></code> - renders the single best match, not just the first.
Route	<pre><Route path="/users/:id" element={<User />} /></pre>	- Defines path-to-component mapping - dynamic segments start with <code>:</code> for parameters.

Component	Example	Description
Link	<code><Link to="/about">About</Link></code>	- Creates navigation links without a full page reload - renders an anchor tag with a client-side click handler.
NavLink	<code><NavLink to="/home" className={({isActive}) => isActive ? 'active' : ''} /></code>	- Like Link but adds active styling when the current route matches - useful for navigation menus.
useNavigate	<code>const navigate = useNavigate() navigate('/dashboard')</code>	- Hook for programmatic navigation - replaces the older <code>useHistory</code> with a simpler API.
useParams	<code>const { id } = useParams()</code>	- Extracts URL parameters from the current route - returns an object of string values keyed by param name.
useLocation	<code>const location = useLocation()</code>	- Returns the current location object with <code>pathname</code>, <code>search</code>, <code>hash</code>, <code>state</code> - useful for tracking page views.
Outlet	<code>function Layout() { return <div><Nav /><Outlet /></div> }</code>	- Renders the matched child route inside the parent - enables nested layouts.
Nested routes	<code><Route path="/users" element={<Users />}> <Route path=":id" element={<User />} /> </Route></code>	- Defines child routes under a parent - relative child paths combine with the parent path.
Router modes	<code><BrowserRouter><App /></BrowserRouter> // vs Data mode: let router = createBrowserRouter([...]) <RouterProvider router={router} /></code>	- React Router works in three additive modes: Declarative (<code><BrowserRouter></code>, <code><Link></code>) for basic routing Data (<code>createBrowserRouter</code>) adds loaders, actions and fetchers Framework wraps Data mode with a Vite plugin for type-safe routes and SSR.
createBrowserRouter	<code>let router = createBrowserRouter([{ path: "/", Component: Root, loader: loadRootData },]); <RouterProvider router={router} /></code>	- Creates a data router that manages the URL via <code>history.pushState</code> / <code>replaceState</code> - passed to <code><RouterProvider></code> to unlock loaders, actions and pending UI.
Loader	<code>loader: async () => { return { records: await getSomeRecords() }; } const { records } = useLoaderData();</code>	- Provides data to a route before its component renders, defined on the route's <code>loader</code> property - read in the component with <code>useLoaderData</code>.
Action	<code>action: async ({ request }) => { let formData = await request.formData(); return updateProject({ title: formData.get("title") }); }</code>	- Handles data mutations defined on a route's <code>action</code> property, triggered by <code><Form method="post"></code> - completing an action revalidates all loader data on the page automatically.
Form	<code><Form action="/events" method="post"> <input name="title" type="text" /> </Form></code>	- A progressively enhanced <code><form></code> that submits to a route's action via fetch - keeps working before JavaScript loads since it is built on the native HTML form.
useFetcher	<code>let fetcher = useFetcher() fetcher.load("/some/route") <fetcher.Form method="post" /></code>	- Loads or submits data without a navigation, tracking its own independent state - useful for polling, optimistic updates, or forms that shouldn't add history entries.
useNavigation	<code>let navigation = useNavigation() navigation.state // "idle", "loading" or "submitting"</code>	Returns the current navigation, defaulting to idle, so a component can render pending UI like a spinner while a route loads or a form submits.

Component	Example	Description
useSearchParams	<pre>const [searchParams, setSearchParams] = useSearchParams() setSearchParams({ tab: "1" })</pre>	- Reads and updates the URL's query string as <code>URLSearchParams</code> - calling the setter causes a navigation to the new URL.
Navigate	<pre><Navigate to="/tasks" /></pre>	- Component-based redirect for class components where Hooks cannot be used <code>useNavigate</code> is recommended everywhere else
TanStack Router	<pre>const route = createRoute({ getParentRoute: () => rootRoute, path: '/users/\$id', })</pre>	- Fully type-safe alternative to React Router - search params, path params, and loader data are all TypeScript-inferred with no manual annotations.

TABLE 19

Styling Approaches

There's no one blessed way to style React, and the options here represent genuinely different philosophies: utility classes with Tailwind, locally scoped CSS Modules, CSS-in-JS libraries like Styled Components and Emotion, preprocessors like Sass, and React's own hoisted `<style>` tag; the inline style prop has its row in JSX and Rendering. Each makes its own tradeoff between speed, scoping, and runtime cost.

Method	Example	Description
Tailwind CSS	<pre><div className="flex items-center gap-4 p-4 bg-white" /></pre>	Utility-first framework: compose predefined single-purpose classes in the markup - rapid development, and one of the most widely adopted React styling choices today
CSS Modules	<pre>import styles from './Button.module.css' <div className={styles.button} /></pre>	Locally scoped CSS by default - build tool hashes class names to unique identifiers to prevent global conflicts
Styled Components	<pre>const Button = styled.button`color: blue;`</pre>	CSS-in-JS library using tagged template literals - styles scoped to the component, with prop-based dynamic styling and theming - entered maintenance mode in 2025
Emotion	<pre><div css={css`color: red;`} /></pre>	- CSS-in-JS supporting both string and object styles - adds a <code>css</code> prop for direct styling, widely used in design systems
Sass/SCSS	<pre>import './styles.scss'</pre>	- CSS preprocessor with variables, nesting, mixins - must be compiled to plain CSS at build time; browsers can't read SCSS directly
Style Precedence & Dedup	<pre><style href="chart-1" precedence="medium"> {stylesheet} </style></pre>	- React hoists <code>style</code> tags to <code>head</code>, deduplicates by href, and orders them by <code>precedence</code> - lets a component ship its own stylesheet without a separate loader

TABLE 20

Build Tools and Ecosystem

Picking how to start a React project matters as much as the code itself. This is the current lay of the land: Vite and Parcel for fast SPAs, Next.js and React Router in Framework Mode as the leading full-stack options, TanStack Start and Expo for newer or mobile targets, plus the bundlers and transpilers underneath. Remix merged into React Router v7 and Create React App is no longer maintained, so both stay here mainly as signposts to what replaced them.

Tool	Example	Description
Vite	<pre>npm create vite@latest my-app -- --template react</pre>	Fast dev server serving your source over native ES modules - bundles production builds with Rollup (now the Rust-based Rolldown), not webpack - recommended for new projects.

Tool	Example	Description
Next.js	<code>npx create-next-app@latest</code>	• Production framework with SSR, SSG, routing, API routes, and React Server Components built-in • most popular production React framework.
React Router (Framework Mode)	<code>npx create-react-router@latest my-react-router-app</code>	• Full-stack framework mode, wraps React Router's Data Mode with a Vite plugin • type-safe routing, SSR, SPA and static rendering strategies • successor to Remix, which merged into it.
TanStack Start	<code>npx @tanstack/cli@latest create</code>	• Full-stack framework built on TanStack Router • type-safe routing, server functions, and SSR • rapidly growing alternative to Next.js.
Expo	<code>npx create-expo-app@latest</code>	• Recommended framework for starting a React Native app • file-based routing, native module library, and EAS build/deploy services • built with the React Native team at Meta.
React Native	<pre>function HomeScreen() { return (<View><Text>Hello World</Text></View>); }</pre>	• Build native mobile apps using React • JavaScript drives real native platform UI, shared across iOS and Android with platform-specific code.
Gatsby	<code>npm init gatsby</code>	• Static site generator with GraphQL data layer • builds HTML pages at build time, excellent for content-heavy sites and blogs.
Webpack	Module bundler that compiles JavaScript and assets; highly configurable but complex setup.	• Flexible build tool used by legacy React projects • builds a dependency graph and combines modules into bundles.
Turbopack	Built into Next.js, run via <code>next dev</code>	• Rust-based incremental bundler by Vercel • built into Next.js as its default bundler • significantly faster cold starts and incremental builds.
Babel	<code>@babel/preset-react</code>	• Transpiles JSX and modern JS to compatible JavaScript • does not bundle modules, that is a separate job.
Parcel	<code>npm create parcel react-client my-react-app</code>	• Zero-config bundler with fast refresh, JSX, TypeScript and Flow support built in • scaffolds a client-only React SPA in one command.
Rsbuid	<code>npx create-rsbuid --template react</code>	• Rspack-powered build tool with tuned defaults and fast refresh • underlies full-stack frameworks like TanStack Start and Modern.js.
Remix (legacy)	<code>npx create-remix@latest</code>	• Framework focused on web fundamentals: nested routing, progressive enhancement, optimistic UI • merged into React Router v7, no longer recommended for new projects.
Create React App (legacy)	<code>npx create-react-app my-app</code>	• Zero-config starter, no longer actively maintained • use Vite or Next.js for new projects.

TABLE 21

State Management Libraries

When `useState` and Context stop scaling, these libraries take over, and they don't all solve the same problem. Redux Toolkit, Zustand, and Jotai handle client state with very different ergonomics, while TanStack Query and SWR specialize in server state: caching, syncing, and revalidating data from APIs. Knowing which bucket your problem falls into is half the decision.

Library	Example	Description
Redux Toolkit	<pre>const store = configureStore({ reducer }) useSelector(state => state.user)</pre>	• Modern Redux with less boilerplate • includes <code>createSlice</code>, <code>createAsyncThunk</code>, opinionated defaults • best for large teams.

Library	Example	Description
Zustand	<pre>const useStore = create(set => ({ count: 0, inc: () => set(s => ({ count: s.count + 1 })) })))</pre>	• Minimal state management with hooks-based API • no providers, boilerplate-free, tiny bundle • most popular lightweight option.
TanStack Query	<pre>const { data } = useQuery({ queryKey: ['user'], queryFn: fetchUser })</pre>	• Server state management with caching, sync, and background updates • not general state but the standard for async data.
SWR	<pre>const { data } = useSWR('/api/user', fetcher)</pre>	• Data fetching hook with caching • "stale-while-revalidate" strategy • lightweight Vercel-maintained alternative to TanStack Query.
Jotai	<pre>const [count, setCount] = useAtom(countAtom)</pre>	• Primitive and flexible atom-based state • bottom-up approach, no string keys, TypeScript-first.
MobX	<pre>const store = observable({ count: 0 }) const Counter = observer(() => <div>{store.count}</div>)</pre>	• Reactive state management using observables • automatic dependency tracking, minimal boilerplate.
Recoil (legacy)	<pre>const [text, setText] = useRecoilState(textState)</pre>	• Facebook's atomic state library with fine-grained subscriptions • not actively maintained (GitHub repo archived Jan 2025), prefer Jotai for similar patterns.

TABLE 22

Accessibility Best Practices

Accessibility isn't a feature you bolt on at the end. It starts with using the right HTML element for the job, so a button is a `<button>` and keyboard support comes for free. These practices cover the rest: labelling inputs, managing focus, the ARIA attributes to use when native semantics fall short, and the contrast ratios that keep content readable for everyone.

Practice	Example	Description
Semantic HTML	<pre><button>Click</button> not <div onClick>Click</div></pre>	• Use correct HTML elements for functionality • provides built-in keyboard support and screen reader context.
Form labels	<pre><label htmlFor="email">Email</label> <input id="email" /></pre>	• Associate labels with inputs using <code>htmlFor</code> • enables clicking label to focus input • required for screen readers.
Focus management	<pre>useEffect(() => { const prev = document.activeElement; dialogRef.current.focus(); return () => prev.focus(); }, []);</pre>	• Programmatically move focus to appropriate elements • essential for modals, route changes, dynamic content • restore focus to the trigger element once a dialog closes.
ARIA attributes	<pre><div role="button" aria-label="Close" /></pre>	• Add accessibility semantics when semantic HTML is insufficient • written with the same hyphenated names as HTML (<code>aria-label</code> , not <code>ariaLabel</code>) • use sparingly, prefer native elements.
aria-label	<pre><button aria-label="Close dialog">x</button></pre>	Provides text alternative for screen readers when visible text is insufficient or absent.
Live regions	<pre><div aria-live="polite" aria-atomic="true">{status}</div></pre>	• Announce dynamic content changes to screen readers • <code>polite</code> waits for pause, <code>assertive</code> interrupts.
Keyboard navigation	<pre>onKeyDown={e => e.code === 'Enter' && doSomething() }</pre>	• Ensure all interactive elements are accessible via keyboard • implement arrow keys for custom widgets.
used for a11y	<pre>const id = useId() <label htmlFor={id}>Name</label> <input id={id} /></pre>	• Generates stable unique IDs for linking labels and inputs • safe in SSR, so it avoids hydration mismatches.

Practice	Example	Description
Skip links	<code>Skip to main</code>	• Bypass navigation links at page top • allows keyboard users to jump directly to content.
Alt text	<code></code>	• Describe image content for screen readers • use empty <code>alt=""</code> for purely decorative images.
Color contrast	Verify with tools like axe or Lighthouse	Minimum 4.5:1 ratio for normal text, 3:1 for large text per WCAG 2.2 AA.

TABLE 23

Advanced React Features

React's concurrent renderer can pause, interrupt and prioritize rendering work, and `startTransition` is the main way to mark an update as non-urgent so it doesn't block the UI. `Activity` and `ViewTransition` build on that same scheduler to hide, pre-render and animate content without losing state, while `use` lets a component read a promise or context directly during render.

Feature	Example	Description
Concurrent rendering	<code>createRoot(el).render(<App />)</code>	• React 18+ feature enabling interruptible rendering • allows React to pause, abort, and prioritize urgent updates.
startTransition	<code>startTransition(async () => { await submitData(form) })</code>	• Marks updates as non-urgent transitions, keeping the UI responsive • supports async functions (React 19) • since 19.3, each transition renders independently instead of being entangled into one.
use hook	<code>const data = use(promise)</code>	• React 19 API that reads resources like promises or context • not a Hook, so uniquely it can be called conditionally unlike other hooks.
Activity component	<code><Activity mode="hidden"> <ExpensivePanel /> </Activity></code>	• React 19.2: hides or shows UI without unmounting, preserving state and DOM • destroys Effects while hidden and re-creates them when visible again • a hidden Activity pre-renders its children at low priority so they're ready sooner.
ViewTransition	<code>{isShowing && (<ViewTransition><Component /></ViewTransition>)}</code>	• Stable since React 19.3: wraps a tree to animate it with the browser's View Transition API • triggers on changes from <code>startTransition</code>, <code>useDeferredValue</code>, or <code>Suspense</code>.
addTransitionType	<code>startTransition(() => { addTransitionType('my-transition-type') setState(newState) })</code>	React 19.3 API that labels the cause of a transition so a nested <code><ViewTransition></code> can style each type differently.

TABLE 24

Server Components and Server Rendering

Server Components render only on the server, shipping no JS for that part of the tree, while `'use client'` and `'use server'` mark the boundary where client code and Server Functions begin. The rest of this table turns that server output into HTML: streaming, hydrating, caching, prerendering, resuming, and hinting the browser about resources to fetch early.

Concept	Example	Description
Server Components	<code>async function Page() { const data = await fetchData() return <Product data={data} /> }</code>	• Async components that run only on the server • fetch data directly and render to HTML, shipping zero JS for that part to the client
'use client' directive	<code>'use client' import { useState } from 'react'</code>	• Marks a module boundary for client-side code in RSC • components below this point ship JS to the browser

Concept	Example	Description
Server Functions	<pre>async function addToCart(data) { 'use server' // ... }</pre>	- Functions marked with <code>'use server'</code> that run only on the server - call them from client components or a form's <code>action</code> prop
renderToPipeableStream	<pre>const { pipe } = renderToPipeableStream(<App />, { bootstrapScripts: ['/main.js'], onShellReady() { pipe(response) } })</pre>	Renders a React tree to HTML as a Node.js Stream , sending the shell first and streaming in Suspense content as it resolves
renderToReadableStream	<pre>const stream = await renderToReadableStream(<App />, { bootstrapScripts: ['/main.js'] })</pre>	Renders a React tree to HTML as a Readable Web Stream , for edge runtimes, Deno and browsers
hydrateRoot	<pre>const root = hydrateRoot(document.getElementById('root'), <App />)</pre>	Attaches React to HTML already rendered on the server, making it interactive
cache	<pre>const getUser = cache(fetchUser) const user = await getUser(id)</pre>	- Memoizes a function's result for Server Components only - the cache resets on every server request
cacheSignal	<pre>import { cache, cacheSignal } from 'react' const dedupedFetch = cache(fetch) await dedupedFetch(url, { signal: cacheSignal() })</pre>	Returns an <code>AbortSignal</code> that aborts once React finishes rendering, to cancel in-flight work no longer needed
Static prerendering (prerender)	<pre>const { prelude } = await prerender(<App />, { bootstrapScripts: ['/main.js'] })</pre>	- Renders a tree to fully static HTML ahead of any request <code>prerenderToNodeStream</code> is the Node.js Stream equivalent
resume	<pre>const stream = await resume(<App />, postponedState)</pre>	Resumes a prerendered tree, streaming in the dynamic parts left out for partial prerendering
browser()	<pre>import { Suspense, use } from 'react' import { browser } from 'react-dom' function Draft() { use(browser('needs window')) return <DraftEditor /> }</pre>	Marks a subtree browser-only inside a <code>Suspense</code> boundary; errors during server rendering, resolves once in the browser (React 19.3)
preload	<pre>preload("https://example.com/font.woff2", { as: "font" })</pre>	Hints the browser to start downloading a resource before it is needed
preinit	<pre>preinit("https://example.com/script.js", { as: "script" })</pre>	Hints the browser to download and execute a script or stylesheet right away
preconnect	<pre>preconnect("https://example.com")</pre>	Hints the browser to open a connection to a server before a resource is requested from it
prefetchDNS	<pre>prefetchDNS("https://example.com")</pre>	- Resolves a server's DNS ahead of time - lighter-weight than <code>preconnect</code> for many speculative domains

References

Official Documentation

1. Functional Component
3. Component composition
5. Importing and Exporting Components
7. Class component (legacy)
9. JSX rules
11. Conditional rendering
13. Inline styling
15. Linking external resources
17. useState
19. useContext
21. useReducer
23. useCallback
25. useFormStatus
27. useTransition
29. useLayoutEffect
31. useEffectEvent
33. useInsertionEffect
35. Local state
37. Updating arrays in state
39. State batching
41. Lifting state up
43. Avoid duplicated state
45. State as a Snapshot
47. Components and Hooks Must Be Pure
49. Your UI as a Tree
51. Event handler
53. Custom events
55. Select box
57. File input
59. Event listeners
61. Removing Effect dependencies
63. Ref cleanup
65. flushSync
67. createRef (legacy)
69. use(Context)
71. Render props
73. Children (legacy)
75. React.memo
77. Suspense
79. React Compiler
81. exhaustive-deps
83. "use memo" / "use no memo" directives
85. Gating
87. target
89. Props interface
91. React Testing Library
93. screen queries
95. fireEvent
97. act()
99. Jest
101. Snapshot testing
103. Profiler
105. captureOwnerStack
107. Routes
109. Link
111. useNavigate
2. Props
4. Fragment
6. createRoot
8. PureComponent (legacy)
10. JSX expression
12. List rendering
14. Document Metadata
16. Portal
18. useEffect
20. useRef
22. useMemo
24. useActionState
26. useOptimistic
28. useDeferredValue
30. useId
32. useSyncExternalStore
34. useDebugValue
36. Updating objects in state
38. Controlled component
40. Derived state
42. Resetting state with a key
44. Rules of Hooks
46. Render and Commit
48. React Calls Components and Hooks
50. Strict Mode
52. Passing an argument
54. Form submission
56. Textarea
58. Custom Hook
60. Reconnecting on dependency change
62. DOM manipulation
64. useImperativeHandle
66. forwardRef (legacy)
68. createContext
70. Context with reducer
72. Higher-Order Component
74. Try-catch in handlers
76. Code splitting
78. Production build
80. eslint-plugin-react-hooks
82. compilationMode
84. Incremental adoption
86. panicThreshold
88. Compiling libraries
90. Generic components
92. render
94. userEvent
96. waitFor
98. renderHook
100. Mock functions
102. React DevTools
104. React Performance Tracks
106. BrowserRouter
108. Route
110. NavLink
112. useParams

- 113. `useLocation`
- 115. `createBrowserRouter`
- 117. `useFetcher`
- 119. `useSearchParams`
- 121. TanStack Router
- 123. Styled Components
- 125. Sass/SCSS
- 127. Vite
- 129. TanStack Start
- 131. Gatsby
- 133. Babel
- 135. TanStack Query
- 137. Form labels
- 139. ARIA attributes
- 141. Live regions
- 143. `startTransition`
- 145. `ViewTransition`
- 147. Server Components
- 149. Server Functions
- 151. `renderToReadableStream`
- 153. `cache`
- 155. Static prerendering (`prerender`)
- 157. `browser()`
- 159. `preinit`
- 161. `prefetchDNS`
- 163. `rules-of-hooks`
- 165. `ViewTransition`
- 167. Multiple contexts - <https://react.dev/learn/passing-data-deeply-with-context#using-and-providing-context-from-the-same-component>
- 169. Prop destructuring - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring_assignment
- 171. Children typing - https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/function_components#react-component-props
- 173. Event typing - https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/forms_and_events
- 175. Ref typing - <https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/hooks#userref>
- 177. Vite - <https://vite.dev/guide/why>
- 114. `Outlet`
- 116. `Form`
- 118. `useNavigation`
- 120. `Navigate`
- 122. Tailwind CSS
- 124. `Emotion`
- 126. Style Precedence & Dedup
- 128. `Next.js`
- 130. `Expo`
- 132. `Turbopack`
- 134. `Rsbuild`
- 136. Semantic HTML
- 138. Focus management
- 140. `aria-label`
- 142. Keyboard navigation
- 144. Activity component
- 146. `addTransitionType`
- 148. `'use client'` directive
- 150. `renderToPipeableStream`
- 152. `hydrateRoot`
- 154. `cacheSignal`
- 156. `resume`
- 158. `preload`
- 160. `preconnect`
- 162. Error Boundary
- 164. Gating
- 166. Console logging - <https://developer.mozilla.org/en-US/docs/Web/API/console>
- 168. Production build - <https://react.dev/learn/start-a-new-react-project#production-grade-react-frameworks>
- 170. `useEffectEvent` - https://react.dev/reference/react/experimental_useEffectEvent
- 172. Context typing - <https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/context>
- 174. Generic components - https://react-typescript-cheatsheet.netlify.app/docs/advanced/patterns_by_usecase#generic-components
- 176. TypeScript - <https://www.typescriptlang.org/docs/handbook/2/basic-types.html>

Specifications & Standards

- 178. Alt text - <https://www.w3.org/WAI/tutorials/images/>
- 179. Color contrast - <https://www.w3.org/WAI/WCAG21/Understanding/contrast-minimum.html>

Source Code & Repositories

- 180. Error boundary library
- 181. List virtualization
- 182. Components tab
- 183. `$r` (console)
- 184. CSS Modules
- 185. Styled Components

Articles & Tutorials

- 186. State colocation
- 188. Form libraries
- 190. Compound components
- 192. Slots pattern
- 194. `Vitest`
- 196. Nested routes
- 187. Event delegation
- 189. Provider pattern
- 191. Container/Presentational
- 193. Ref typing
- 195. Source maps
- 197. Router modes

- 198. Loader
- 200. React Native
- 202. Parcel
- 204. Create React App (legacy)
- 206. Zustand
- 208. Jotai
- 210. Recoil (legacy)
- 212. Alt text
- 214. startTransition
- 216. TanStack Router - <https://tanstack.com/router/latest>
- 218. Compound components -
<https://kentcdodds.com/blog/compound-components-with-react-hooks>
- 220. CSS-in-JS - <https://cssinjs.org/>
- 222. Interval/Timeout -
<https://overreacted.io/making-setinterval-declarative-with-react-hooks/>
- 224. MobX - <https://mobx.js.org/>
- 199. Action
- 201. Webpack
- 203. Remix (legacy)
- 205. Redux Toolkit
- 207. SWR
- 209. MobX
- 211. Skip links
- 213. Color contrast
- 215. Next.js - <https://nextjs.org/docs>
- 217. TanStack Start - <https://tanstack.com/start/latest>
- 219. Context performance -
<https://kentcdodds.com/blog/how-to-optimize-your-context-value>
- 221. ESLint - <https://eslint.org/>
- 223. List virtualization - <https://react-window.vercel.app/>